

МАНИФЕСТ

Данные по шагам 1-5

Шаг 1. Постановка цели.

Целью данной работы является создания ML-системы, обеспечивающей корректный расчёт складских запасов без создания сценариев дефицита или переизбытка товара. В качестве таргета в задаче принимается предсказание спроса на 7 дней вперёд.

ML-система должна принимать решения на основании данных трёх таблиц: "products.csv", "warehouses.csv", "orders.csv".

В качестве бизнес-метрики для данной системы может быть принято общее число случаев, когда заказанного товара не хватило (т.н. Stockout Rate). Для ML-задачи этой метрике соответствует минимизация ошибки прогноза по метрике MAE. Таки образом, например, модель допускалась бы в продакшн, если MAE ниже заданного порога. Прогноз ML-модели может быть использован для расчёта рекомендованного запаса, например:

$$\text{recommended_stock} = \text{predicted_demand_next_7d} + \text{safety_stock}.$$

Шаг 2. Выбор уровня сложности реализуемой системы.

1. Предыстория

Объективной целью заказчика является максимизация прибыли от бизнеса, в данном случае – от ретейла. Одной из проблем на пути реализации этой цели является неверный расчёт складских запасов, которую и призвана решить данная ML-система.

2. Ценностное предложение

Разрабатываемый продукт призван до минимума сократить издержки заказчика, связанные с неверным расчётом складских запасов:

- упущенной выгодой при их недооценке,
- потерями на хранение избыточного количества товара и истекающими сроками годности при переоценке запасов.

3. Цели

- Создание пайплайна автоматической подготовки данных, обучения и валидации модели (Automated pipeline).
- Создание технических средств взаимодействия с моделью для инференса в продакшене.

- Создание системы мониторинга качества модели и системы её переобучения, с автоматизацией принятия решения о замене продакшн-версии модели.

4. Решение

Для достижения поставленных целей принимается решение реализовывать ML-систему уровня зрелости 2.

В рамках данной системы предполагается реализовать следующий функционал:

- Orchestrated experiment – реализуется в Airflow DAG. DAG запускает полный цикл: data extraction, data validation, data preparation, model training, model evaluation, model validation. Также он параллельно пишет результаты в MLflow.

- CI: Build, test, package – реализуется в GitHub Actions ci.yml. Проверяет код, тесты, собирает Docker-образы.

- CD: Pipeline deployment – реализуется в GitHub Actions deploy.yml, deploy_keep_alive.yml. Поднимает инфраструктуру через Terraform. Настраивает виртуальную машину (VM) через Ansible. Разворачивает сервисы через Docker Compose.

- Automated pipeline – реализуется как Python-пайплайн совместно с Airflow DAG. Логика работы этого пайплайна близка к Orchestrated experiment, но, в общем случае Orchestrated experiment может проходить при ручном вмешательстве дата-сайентиста, а Automated pipeline должен быть полностью автономным.

- CD: Model serving – реализуется через FastAPI + blue/green deployment.

- Есть API: POST(/predict), GET (/health, /metrics). API предназначен, чтобы принимать сырые таблицы "products.csv", "warehouses.csv", "orders.csv".

- Performance monitoring – реализуется через Prometheus + Grafana + Evidently. Пишутся инциденты и метрики. Делается drift detection. Есть alert для ошибок батч-предсказаний.

- Хранение данных – реализуется в MinIO (сырые csv, подготовленные train/test/reference, модели, отчёты evidently), PostgreSQL (метаданные Airflow), MLflow (метаданные экспериментов и артефакты), локально на VM (временные файлы пайплайна, docker volumes).

В рамках ML-системы не планируется реализовывать:

- Feature store, т.к. для поставленной задачи достаточно прочих способов хранения данных;

- Data analysis – частично заменён автоматическим data validation;
- Model analysis – не реализована возможность ручного участия в Orchestrated experiment.

5. Осуществимость

За время выполнения задания был реализован прототип системы, содержащий контейнеры всех заявленных сервисов. Была протестирована работоспособность расчётных скриптов локально, затем протестировано создание ВМ с помощью terraform-документации и развёртывание на ней всех контейнеров с помощью ansible:

```
TASK [Print docker-compose status after start] *****
ok: [111.88.254.251] => {
  "compose_ps_after_start.stdout_lines": [
    "-----",
    "Name                                Command                                State          Ports",
    "-----",
    "airflow_init                        sh -ec airflow db                    Exit 0",
    "airflow_scheduler                  sh -ec while [ ! -f                  Up             5000/tcp, 8000/tcp, 8080/tcp",
    "airflow_scheduler                  /opt/a ...",
    "airflow_webserver                  sh -ec while [ ! -f                  Up             5000/tcp, 8000/tcp, 0",
    "airflow_webserver                  /opt/a ...",
    "grafana                            /run.sh                               Up             0.0.0.0:3000->3000/tc",
    "grafana                            p,:::3000->3000/tcp",
    "minio                              /usr/bin/docker-                     Up (healthy)   0.0.0.0:9000->9000/tc",
    "minio                              entrypoint ...",
    "minio                              p,:::9000->9000/tcp, ",
    "minio                              0.0.0.0:9001->9001/tc",
    "minio                              p,:::9001->9001/tcp",
    "minio_init                        python                                Exit 0",
    "mlflow                             scripts/init_minio.py",
    "mlflow                             sh -c mlflow server                  Up (healthy)   0.0.0.0:5000->5000/tc",
    "mlflow                             --host ...",
    "mlflow                             p,:::5000->5000/tcp, ",
    "mlflow                             8000/tcp, 8080/tcp",
    "postgres                           docker-entrypoint.sh                 Up (healthy)   0.0.0.0:5432->5432/tc",
    "postgres                           postgres",
    "prometheus                         /bin/prometheus                      Up             0.0.0.0:9090->9090/tc",
    "prometheus                         --config.f ...",
    "prometheus                         p,:::9090->9090/tcp",
    "warehouse_api                     uvicorn app.main:app                  Up             5000/tcp, 0.0.0.0:80-",
    "warehouse_api                     --hos ...",
    "warehouse_api                     >8000/tcp,:::80->8000",
    "warehouse_api                     /tcp, 8080/tcp",
    ]
}
```

За оставшейся время была выполнена автоматизация CI/CD процессов в GitHub Actions, с автоматическим созданием ВМ и последующим выполнением .github/workflows/deploy.yml. В deploy.yml реализовано автоматическое удаление ВМ после завершения процесса и оно успешно выполняется – с результатами прошу ознакомиться в репозитории: <https://github.com/Roman-Golubev/warehouse-mlops.git>. На данный момент у меня не хватило времени/компетенций, чтобы отладить работу ansible из deploy.yml в CI/CD – в Actions задача завершается с ошибкой, но это не препятствует автоматическому удалению машины.

6. Данные

Для генерации данных были созданы специальные скрипты: `generate_test_csv.py` и `data_pipeline.py` (метод `generate_synthetic_data`). Этими скриптами создаются все три таблицы проекта. Метод `data_preparation` из `data_pipeline.py` объединяет данные этих таблиц в таблицу признаков, добавляет новые признаки и разделяет на обучающие и тестовые датасеты с учётом хронологии. При этом выделяется обучающая и тестовая серии таргета – прогноза запасов на неделю.

7. Метрики

В валидационном скрипте модели `validate.py` предусмотрена оценка по метрикам с пороговыми значениями:

```
mae_ok = candidate_metrics["mae"] <= production_metrics["mae"] * 1.03;  
rmse_ok = candidate_metrics["rmse"] <= production_metrics["rmse"] * 1.03;  
r2_ok = candidate_metrics["r2"] >= production_metrics["r2"] - 0.02.
```

При этом переключение на новую модель выполняется этим же скриптом - `blue/green` переключатель.

В скрипте `drift.py` реализована оценка дрифта по признакам и детекция снижения качества данных с помощью библиотеки `evidently` применительно к паре датасетов `reference.csv` (бейслайновый), `test.csv` (тестовый). Обращение к `validate.py` и `drift.py` реализовано в DAG `retrain_pipeline.py`.

Полученные метрики публикуются в Prometheus и визуализируются в Grafana. В Grafana предполагалась реализация алерта `prediction_request_errors_total` – рост числа ошибок предсказаний на батчах.

Из перечисленных метрик на язык бизнеса проще всего переводится `mae_ok`.

8. Оценка качества модели

Как было указано ранее, она выполняется в `validate.py` по параметрам: `mae_ok`, `rmse_ok`, `r2_ok`.

9. Подбор модели

На данной стадии развития проекта была принята модель `RandomForestRegressor`. На сгенерированных данных её результаты соответствовали установленным порогам по `mae_ok`, `rmse_ok`, `r2_ok`; что позволяло выводить её в продакшн.

10. Инференс

Постановка задачи не требует онлайн обучения модели, т.к. предсказание выполняется на неделю вперёд и, основываясь на этом предсказании,

организационные решения о пополнении запасов должны приниматься лишь раз в несколько дней. API был спроектирован так, чтобы принимать на вход три таблицы, о чём написано выше.

11. Обратная связь

Обратная связь в модели реализуется сервисом AirFlow его DAG выполняет цепочку действий: `data_pipeline >> model_training >> model_drift >> model_validation`. Обратная связь в виде метрик – это база для автоматического принятия решения по переобучению модели, что соответствует MDD-подходу.

12. Управление проектом

В идеальных условиях полноценная команда для реализации этого проекта должна состоять из трёх человек:

- дата-сайентиста, выполняющего ручные эксперименты и подбирающего наилучшую модель и её гиперпараметры;
- DVOps-инженера, обеспечивающего реализацию инфраструктуры проекта в соответствии с IaC;
- программиста, адаптирующего скрипты проекта под решения дата-сайентиста и DVOps-инженера.

При достаточном бюджете к проекту целесообразно привлечь UX/UI специалиста, так как необходимо сделать продукт визуально приятным для заказчика.

Уверен, что такая команда способно выполнить проект за неделю. Я свой прототип подготовил за 3 рабочих дня, при этом я с нуля изучал создание VM через terraform (ранее не приходилось) и работу в Yandex Cloud.

Шаг 3. Создание ML-системы.

С файлами разработанного прототипа системы прошу ознакомиться по ссылке: <https://github.com/Roman-Golubev/warehouse-mlops.git>.

Шаг 4. Управление рисками.

Для полноценного продукта, который должен стать результатом труда обозначенного выше коллектива, трёхуровневая система индикации и контроля за состоянием ML-системы может иметь следующие составляющие:

Уровень бизнеса (данных):

1. Доступность пайплайна данных (ingestion_rate):
 - SLI: uptime_ingestion_rate,
 - SLO: 99,9 % за месяц,
 - SLA: менее 99,0 % в течение двух недель подряд -> оператор.
2. Задержка между событием и его попаданием в хранилище (data freshness / latency):
 - SLI: latency_minutes (время от события до появления в feature store),
 - SLO: <= 15 минут для 95 % событий,
 - SLA: более 30 минут более чем 2 часа подряд -> оператор.
3. Полнота данных:
 - SLI: completeness_rate,
 - SLO: >= 99 % в каждой загрузке набора,
 - SLA: < 97 % в течение 12 часов -> оператор.
4. Качество данных:
 - SLI: ошибки на миллион записей (error_rate_ppm),
 - SLO: <= 100 ppm,
 - SLA: > 500 ppm в течение 6 часов -> оператор.
5. Дрейф схемы и данных:
 - SLI: drift_score,
 - SLO: менее 5% по ключевым признакам за месяц,
 - SLA: > 15% -> оператор.

Уровень кода:

1. Успешность развёртываний:
 - SLI: доля успешных релизов (deployment_success_rate),
 - SLO: >= 99,5 % релизов в месяц,
 - SLA: менее 98 % за два релиза подряд -> оператор.
2. Время выполнения пайплайна:
 - SLI: время от коммита до развёртывания в окружении,
 - SLO: <= 20 минут,
 - SLA: > 60 минут для 3 последовательных запусков -> оператор.
3. Покрытие тестами:
 - SLI: test_coverage_percent,
 - SLO: >= 85 %,
 - SLA: < 75 % на двух сборках подряд -> оператор.

4. Воспроизводимость обучения:

- SLI: воспроизводимость (детерминированность) результатов обучающих запусков
- SLO: 100% воспроизводимость при заданном seed,
- SLA: обнаружен недетерминизм в двух запусках -> оператор.

5. Дрейф модели:

- SLI: процент ухудшения AUC, ассурасу и т.д.,
- SLO: дрейф менее 5 % за месяц,
- SLA: дрейф > 15 % -> оператор.

6. Версионирование данных и артефактов:

- SLI: полнота версионирования артефактов пайплайна,
- SLO: 99 % артефактов версионированы,
- SLA: менее 95 % -> оператор.

Уровень инфраструктуры:

1. Доступность ВМ и контейнеров:

- SLI: availability_percent (uptime за период),
- SLO: >= 99.9% в месяц,
- SLA: < 99.5% в месяц -> оператор.

2. Располагаемые ресурсы:

- SLI: загрузка CPU и использование памяти,
- SLO: среднее CPU < 70%, память < 80%,
- SLA: > 90% в течение 15 минут -> оператор.

3. Располагаемое место на диске:

- SLI: свободное место на диске,
- SLO: >= 20 %,
- SLA: < 10 % более чем 24 часа -> оператор.

4. Доступность внешних сервисов:

- SLI: задержки (latency) до ключевых источников данных и сервисов,
- SLO: <= 100 ms 95% времени,
- SLA: > 200 ms более 15 минут -> оператор.

Шаг 5. Принятие решений по Metrics Driven Development.

Решение изложено в ноутбуке MDD_step_5.ipynb.